

[Hide navigation sidebar](#)

[Hide table of contents sidebar](#)

[Toggle site navigation sidebar](#)

[LLVM](#)

[Toggle Light / Dark / Auto color theme](#)

[Toggle table of contents sidebar](#)



[LLVM](#)



- [Frequently Asked Questions \(FAQ\)](#)
- [The LLVM Lexicon](#)

- [Getting Started/Tutorials](#)

[Toggle navigation of Getting Started/Tutorials](#)

- [Architecture & Platform Information for Compiler Writers](#)
- [Performance Tips for Frontend Authors](#)
- [Getting Started with the LLVM System](#)
- [Getting Started with the LLVM System using Microsoft Visual Studio](#)
- [LLVM Programmer's Manual](#)
- [Debugging LLVM](#)
- [LLVM Tutorial: Table of Contents](#)

[Toggle navigation of LLVM Tutorial: Table of Contents](#)

- [My First Language Frontend with LLVM Tutorial](#)
 - [1. Kaleidoscope: Kaleidoscope Introduction and the Lexer](#)
 - [2. Kaleidoscope: Implementing a Parser and AST](#)
 - [3. Kaleidoscope: Code generation to LLVM IR](#)
 - [4. Kaleidoscope: Adding JIT and Optimizer Support](#)
 - [5. Kaleidoscope: Extending the Language: Control Flow](#)
 - [6. Kaleidoscope: Extending the Language: User-defined Operators](#)
 - [7. Kaleidoscope: Extending the Language: Mutable Variables](#)
 - [8. Kaleidoscope: Compiling to Object Code](#)
 - [9. Kaleidoscope: Adding Debug Information](#)
 - [10. Kaleidoscope: Conclusion and other useful LLVM tidbits](#)
 - [1. Building a JIT: Starting out with KaleidoscopeJIT](#)
 - [2. Building a JIT: Adding Optimizations – An introduction to ORC Layers](#)
 - [3. Building a JIT: Per-function Lazy Compilation](#)
 - [4. Building a JIT: Extreme Laziness - Using LazyReexports to JIT from ASTs](#)
 - [MyFirstTypoFix](#)
- [Reference](#)

[Toggle navigation of Reference](#)

- [How To Use Attributes](#)
- [LLVM Command Guide](#)

[Toggle navigation of LLVM Command Guide](#)

- [FileCheck - Flexible pattern matching file verifier](#)
- [clang-tblgen - Description to C++ Code for Clang](#)
- [dsymutil - manipulate archived DWARF debug symbol files](#)
- [lit - LLVM Integrated Tester](#)
- [llc - LLVM static compiler](#)
- [lldb-tblgen - Description to C++ Code for LLDB](#)
- [lli - directly execute programs from LLVM bitcode](#)

- [llubi - LLVM UB-aware Interpreter](#)
- [llvm-addr2line - a drop-in replacement for addr2line](#)
- [llvm-ar - LLVM archiver](#)
- [llvm-as - LLVM assembler](#)
- [llvm-bcanalyzer - LLVM bitcode analyzer](#)
- [llvm-calc-occupancy - AMDGPU occupancy calculator](#)
- [llvm-cgdata - LLVM CodeGen Data Tool](#)
- [llvm-config - Print LLVM compilation options](#)
- [llvm-cov - emit coverage information](#)
- [llvm-cxxfilt - LLVM symbol name demangler](#)
- [llvm-cxxmap - Mangled name remapping tool](#)
- [llvm-debuginfo-analyzer - Print a logical representation of low-level debug information.](#)
- [llvm-diff - LLVM structural 'diff'](#)
- [llvm-dis - LLVM disassembler](#)
- [llvm-dwarfdump - dump and verify DWARF debug information](#)
- [llvm-dwarfutil - A tool to copy and manipulate debug info](#)
- [llvm-exegesis - LLVM Machine Instruction Benchmark](#)
- [llvm-extract - extract a function from an LLVM module](#)
- [llvm-extract-bundle-entry - extract an offload bundle entry](#)
- [llvm-ifs - shared object stubbing tool](#)
- [llvm-install-name-tool - LLVM tool for manipulating install-names and rpaths](#)
- [llvm-ir2vec - IR2Vec and MIR2Vec Embedding Generation Tool](#)
- [llvm-lib - LLVM lib.exe compatible library tool](#)
- [llvm-libtool-darwin - LLVM tool for creating libraries for Darwin](#)
- [llvm-link - LLVM bitcode linker](#)
- [llvm-lipo - LLVM tool for manipulating universal binaries](#)
- [llvm-locstats - calculate statistics on DWARF debug location](#)
- [llvm-mc - LLVM Machine Code Playground](#)
- [llvm-mca - LLVM Machine Code Analyzer](#)
- [llvm-nm - list LLVM bitcode and object file's symbol table](#)
- [llvm-objcopy - object copying and editing tool](#)
- [llvm-objdump - LLVM's object file dumper](#)
- [llvm-offload-binary - LLVM Offload Binary Packager](#)
- [llvm-opt-report - generate optimization report from YAML](#)
- [llvm-otool - Mach-O dumping tool](#)
- [llvm-pdbutil - PDB File forensics and diagnostics](#)
- [llvm-profdata - Profile data tool](#)
- [llvm-profgen - LLVM SPGO profile generation tool](#)
- [llvm-ranlib - generates an archive index](#)
- [llvm-readelf - GNU-style LLVM Object Reader](#)
- [llvm-readobj - LLVM Object Reader](#)
- [llvm-reduce - LLVM automatic testcase reducer.](#)
- [llvm-remarkutil - Remark utility](#)
- [llvm-size - print size information](#)
- [llvm-stress - generate random .ll files](#)
- [llvm-strings - print strings](#)
- [llvm-strip - object stripping tool](#)
- [llvm-symbolizer - convert addresses into source code locations](#)
- [llvm-tblgen - Target Description to C++ Code for LLVM](#)
- [llvm-test-mustache-spec - LLVM tool to test Mustache library compliance](#)
- [llvm-tli-checker - TargetLibraryInfo vs library checker](#)
- [mlir-tblgen - Description to C++ Code for MLIR](#)
- [opt - LLVM optimizer](#)
- [tblgen - Description to C++ Code](#)
- [Using -opt-bisect-limit to debug optimization errors](#)

- [Symbolizer Markup Format](#)
- [The PDB File Format](#)

Toggle navigation of The PDB File Format

- [The MSF File Format](#)
- [The PDB Info Stream \(aka the PDB Stream\)](#)
- [The PDB TPI and IPI Streams](#)
- [The PDB DBI \(Debug Info\) Stream](#)
- [The Module Information Stream](#)
- [The PDB Public Symbol Stream](#)
- [The PDB Global Symbol Stream](#)
- [The PDB Serialized Hash Table Format](#)
- [Introduction](#)
- [On-Disk Format](#)
- [Present and Deleted Bit Vectors](#)
- [CodeView Symbol Records](#)
- [CodeView Type Records](#)
- [Garbage Collection with LLVM](#)
- [Garbage Collection Safepoints in LLVM](#)
- [libFuzzer – a library for coverage-guided fuzz testing.](#)
- [Fuzzing LLVM libraries and tools](#)
- [LLVM Language Reference Manual](#)
- [LLVM IR Undefined Behavior \(UB\) Manual](#)
- [Design and Usage of the InAlloca Attribute](#)
- [LLVM Bitcode File Format](#)
- [Machine IR \(MIR\) Format Reference Manual](#)
- [Global Instruction Selection](#)

Toggle navigation of Global Instruction Selection

- [Generic Machine IR](#)
- [Generic Opcodes](#)
- [MIR Patterns in TableGen](#)
- [Core Pipeline](#)
- [Porting GlobalISel to A New Target](#)
- [Resources](#)
- [IRTranslator](#)
- [Legalizer](#)
- [RegBankSelect](#)
- [InstructionSelect](#)
- [Known Bits Analysis](#)
- [Convergent Operation Semantics](#)
- [LLVM Testing Infrastructure Guide](#)

Toggle navigation of LLVM Testing Infrastructure Guide

- [test-suite Guide](#)
- [GWP-ASan](#)
- [XRay Instrumentation](#)
- [Debugging with XRay](#)
- [FaultMaps and implicit checks](#)
- [LLVM Atomic Instructions and Concurrency Guide](#)
- [Exception Handling in LLVM](#)
- [LLVM Extensions](#)

Toggle navigation of LLVM Extensions

- [How to set up LLVM-style RTTI for your class hierarchy](#)
- [LLVM Block Frequency Terminology](#)
- [LLVM Branch Weight Metadata](#)
- [The Often Misunderstood GEP Instruction](#)
- [Scudo Hardened Allocator](#)
- [Memory Model Relaxation Annotations](#)

- [MemTagSanitizer](#)
- [Dependence Graphs in LLVM](#)
- [Speculative Load Hardening](#)
- [Segmented Stacks in LLVM](#)
- [LLVM's Optional Rich Disassembly Output](#)
- [Stack maps and patch points in LLVM](#)
- [Coroutines in LLVM](#)
- [Pointer Authentication](#)
- [YAML I/O](#)
- [Convergence And Uniformity](#)
- [Machine Learning - Guided Optimization \(MLGO\)](#)
- [Content Addressable Storage](#)
- [LLVM CI Best Practices](#)
- [LLVM GitHub Actions Runners](#)
- [LLVM AI Tool Use Policy](#)
- [.llvm.callgraph Section Layout](#)
- [LLVM Interface Export Annotations](#)
- [LLVM PC Sections Metadata](#)
- [LLVM Qualification Group](#)
- [LLVM Security Response Group](#)
- [LLVM Security Group Transparency Reports](#)
- [System Library](#)
- [Code Transformation Metadata](#)
- [Type Metadata](#)
- [XRay Flight Data Recorder Trace Format](#)
- [User Guides](#)
 - Toggle navigation of User Guides
 - [How To Build On ARM](#)
 - [How To Build Clang and LLVM with Profile-Guided Optimizations](#)
 - [How to cross-compile Clang/LLVM using Clang/LLVM](#)
 - [LLVM Code Coverage Mapping Format](#)
 - [Control Flow Verification Tool Design Document](#)
 - [Building a Distribution of LLVM](#)
 - [Building LLVM with CMake](#)
 - [A guide to Dockerfiles for building LLVM](#)
 - [Support Library](#)
 - [Advanced Build Configurations](#)
 - [Writing an LLVM Pass](#)
 - [Writing an LLVM Pass \(legacy PM version\)](#)
 - [LLVM's Analysis and Transform Passes](#)
 - Toggle navigation of LLVM's Analysis and Transform Passes
 - [KernelInfo](#)
 - [Loop Fusion in LLVM](#)
 - [Stack Safety Analysis](#)
 - [MergeFunctions pass, how it works](#)
 - [LLVM Alias Analysis Infrastructure](#)
 - [MemorySSA](#)
 - [MemProf: Memory Profiling for LLVM](#)
 - [LLVM Loop Terminology \(and Canonical Forms\)](#)
 - [LLVM Cycle Terminology](#)
 - [Auto-Vectorization in LLVM](#)
 - Toggle navigation of Auto-Vectorization in LLVM
 - [Vectorization Plan](#)
 - [The Sandbox Vectorizer](#)
 - [LLVM Link Time Optimization: Design and Implementation](#)
 - [DTLTO](#)

- Toggle navigation of DTLTO
- [The LLVM gold plugin](#)
- [Remarks](#)
- [Source Level Debugging with LLVM](#)
- Toggle navigation of Source Level Debugging with LLVM
 - [Debug Info Assignment Tracking](#)
- [How to Update Debug Info: A Guide for LLVM Pass Authors](#)
- [The LLVM Instrumentor Pass](#)
- [Instruction referencing for debug info](#)
- [Debug info migration: From intrinsics to records](#)
- [Key Instructions debug info in LLVM and Clang](#)
- [Instrumentation Profile Format](#)
- [InstCombine contributor guide](#)
- [Writing an LLVM Backend](#)
- Toggle navigation of Writing an LLVM Backend
 - [How To Use Instruction Mappings](#)
- [The LLVM Target-Independent Code Generator](#)
- [TableGen Overview](#)
- Toggle navigation of TableGen Overview
 - [TableGen BackEnds](#)
 - [1 TableGen Backend Developer's Guide](#)
 - [1 TableGen Programmer's Reference](#)
- [MCJIT Design and Implementation](#)
- [ORC Design and Implementation](#)
- [JITLink and ORC's ObjectLinkingLayer](#)
- [Debugging JIT-ed Code](#)
- [CommandLine 2.0 Library Manual](#)
- [Extending LLVM: Adding instructions, intrinsics, types, etc.](#)
- [How To Add A Constrained Floating-Point Intrinsic](#)
- [How to build Windows Itanium applications.](#)
- [How to Cross Compile Compiler-rt Builtins For Arm](#)
- [Using ARM NEON instructions in big-endian mode](#)
- [Support for AArch64 Scalable Matrix Extension in LLVM](#)
- [Compiling CUDA with clang](#)
- [User Guide for NVPTX Back-end](#)
- [User Guide for AMDGPU Backend](#)
- Toggle navigation of User Guide for AMDGPU Backend
 - [Syntax of AMDGPU Instruction Modifiers](#)
 - [Syntax of AMDGPU Instruction Operands](#)
 - [AMDGPU Instruction Syntax](#)
 - [AMDGPU Instructions Notation](#)
 - [DWARF Extensions For Heterogeneous Debugging](#)
 - [Allow Location Descriptions on the DWARF Expression Stack](#)
 - [Developer Guideline for AMDGPU](#)
- [AMDGPU Asynchronous Operations](#)
- [AMDGPU DMA Operations](#)
- [AMDGPU Execution Synchronization](#)
- [AMDGPU Memory Model](#)
- [User Guide for SPIR-V Target](#)
- Toggle navigation of User Guide for SPIR-V Target
- [User Guide for the DirectX Target](#)
- Toggle navigation of User Guide for the DirectX Target
 - [DirectX Container](#)
 - Toggle navigation of DirectX Container
 - [Architecture and Design of DXIL Support in LLVM](#)
 - Toggle navigation of Architecture and Design of DXIL Support in LLVM

- [Specification of DXIL Operations using TableGen Representation](#)
- [DXIL Resource Handling](#)
Toggle navigation of DXIL Resource Handling
- [Root Signatures](#)
Toggle navigation of Root Signatures
- [Semantic Signatures](#)
- [User Guide for RISC-V Target](#)
- [RISC-V Vector Extension](#)
- [Scheduling Information for RISC-V VCIX Instructions](#)
- [Sandbox IR: A transactional layer over LLVM IR](#)
- [Telemetry framework in LLVM](#)
Toggle navigation of Telemetry framework in LLVM
- [Lightweight Fault Isolation \(LFI\) in LLVM](#)
- [LLVM Admin Tasks](#)
- [Benchmarking tips](#)
- [CMake Primer](#)
- [LLVM Community Code of Conduct](#)
- [FatLTO](#)
Toggle navigation of FatLTO
- [LLVM GitHub User Guide](#)
- [Markdown Quickstart Template](#)
- [Misexpect](#)
Toggle navigation of Misexpect
- [Opaque Pointers](#)
- [Using the New Pass Manager](#)
- [LLVM CoC Incident Reporting Guide](#)
- [Response Guide](#)
- [TableGen Fundamentals](#)
- [yaml2obj](#)
- [Discourse Migration Guide](#)
- [Getting Involved](#)
Toggle navigation of Getting Involved
 - [Contributing to LLVM](#)
 - [LLVM Developer Policy](#)
 - [LLVM Code-Review Policy and Practices](#)
 - [LLVM Community Support Policy](#)
 - [Sphinx Quickstart Template](#)
 - [How to submit an LLVM bug report](#)
 - [LLVM Bug Life Cycle](#)
 - [LLVM Coding Standards](#)
 - [Bisecting LLVM code](#)
 - [Policies on git repositories](#)
 - [Creating an LLVM Project](#)
 - [How To Release LLVM To The Public](#)
 - [How To Validate a New Release](#)
 - [How To Add Your Build Configuration To LLVM Buildbot Infrastructure](#)
 - [LLVM 24.0.0git Release Notes](#)
 - [How to start LLVM Social in your town](#)
 - [Moving LLVM Projects to GitHub](#)
 - [Test-Suite Extensions](#)
 - [Variable Names Plan](#)
 - [Vector Predication Roadmap](#)
- [Request For Comment \(RFC\) process](#)
- [Project Governance](#)

[Back to top](#)

[Edit this page](#)

Toggle Light / Dark / Auto color theme

Toggle table of contents sidebar

LLVM Coding Standards#

Introduction#

This document describes coding standards that are used in the LLVM project. Although no coding standards should be regarded as absolute requirements to be followed in all instances, coding standards are particularly important for large-scale code bases that follow a library-based design (like LLVM).

While this document may provide guidance for some mechanical formatting issues, whitespace, or other “microscopic details”, these are not fixed standards. Always follow the golden rule:

If you are extending, enhancing, or bug fixing already implemented code, use the style that is already being used so that the source is uniform and easy to follow.

Note that some code bases (e.g. `libc++`) have special reasons to deviate from the coding standards. For example, in the case of `libc++`, this is because the naming and other conventions are dictated by the C++ standard.

There are some conventions that are not uniformly followed in the code base (e.g. the naming convention). This is because they are relatively new, and a lot of code was written before they were put in place. Our long-term goal is for the entire codebase to follow the convention, but we explicitly *do not* want patches that do large-scale reformatting of existing code. On the other hand, it is reasonable to rename the methods of a class if you’re about to change it in some other way. Please commit such changes separately to make code review easier.

The ultimate goal of these guidelines is to increase the readability and maintainability of our common source base.

Languages, Libraries, and Standards#

Most source code in LLVM and other LLVM projects using these coding standards is C++ code. There are some places where C code is used either due to environment restrictions, historical restrictions, or due to third-party source code imported into the tree. Generally, our preference is for standards-conforming, modern, and portable C++ code as the implementation language of choice.

For automation, build systems, and utility scripts, Python is preferred and is widely used in the LLVM repository already.

C++ Standard Versions#

Unless otherwise documented, LLVM subprojects are written using standard C++17 code and avoid unnecessary vendor-specific extensions.

Nevertheless, we restrict ourselves to features which are available in the major toolchains supported as host compilers (see [Getting Started with the LLVM System](#) page, section [Software](#)).

Each toolchain provides a good reference for what it accepts:

- Clang: https://clang.llvm.org/cxx_status.html
 - libcxx: <https://libcxx.llvm.org/Status/Cxx17.html>
- GCC: <https://gcc.gnu.org/projects/cxx-status.html#cxx17>
 - libstdc++: <https://gcc.gnu.org/onlinedocs/libstdc++/manual/status.html#status.iso.2017>
- MSVC: <https://learn.microsoft.com/cpp/overview/visual-cpp-language-conformance>

Additionally, there are compiler comparison tables of supported C++ features on cpreference.com.

To keep track with the evolution of the standard, newer C++ versions can be used to build LLVM. However, our support focuses on the minimum supported C++ version and a very recent standard may not yet be supported, or only using the latest version of the supported toolchains and possibly not across all the subprojects.

C++ Standard Library#

Instead of implementing custom data structures, we encourage the use of C++ standard library facilities or LLVM support libraries whenever they are available for a particular task. LLVM and related projects emphasize and rely on the standard library facilities and the LLVM support libraries as much as possible.

LLVM support libraries (for example, [ADT](#)) implement specialized data structures or functionality missing in the standard library. Such libraries are usually implemented in the `llvm` namespace and follow the expected standard interface when there is one.

When both C++ and the LLVM support libraries provide similar functionality, and there isn't a specific reason to favor the C++ implementation, it is generally preferable to use the LLVM library. For example, `llvm::DenseMap` should almost always be used instead of `std::map` or `std::unordered_map`, and `llvm::SmallVector` should usually be used instead of `std::vector`.

We explicitly avoid some standard facilities, like the I/O streams, and instead use LLVM's streams library ([raw_ostream](#)). More detailed information on these subjects is available in the [LLVM Programmer's Manual](#).

For more information about LLVM's data structures and the tradeoffs they make, please consult [that section of the programmer's manual](#).

Python version and Source Code Formatting#

The current minimum version of Python required is documented in the [Getting Started with the LLVM System](#) section. Python code in the LLVM repository should only use language features available in this version of Python.

The Python code within the LLVM repository should adhere to the formatting guidelines outlined in [PEP 8](#).

For consistency and to limit churn, code should be automatically formatted with the [black](#) utility, which is PEP 8 compliant. Use its default rules. For example, avoid specifying `--line-length` even though it does not default to 80. The default rules can change between major versions of black. In order to avoid unnecessary churn in the formatting rules, we currently use black version 23.x in LLVM.

When contributing a patch unrelated to formatting, you should format only the Python code that the patch modifies. For this purpose, use the [darker](#) utility, which runs default black rules over only the modified Python code. Doing so should ensure the patch will pass the Python format checks in LLVM's pre-

commit CI, which also uses darker. When contributing a patch specifically for reformatting Python files, use black, which currently only supports formatting entire files.

Here are some quick examples, but see the black and darker documentation for details:

```
$ pip install black==23.* darker # install black 23.x and darker
$ darker test.py                  # format uncommitted changes
$ darker -r HEAD^ test.py         # also format changes from last commit
$ black test.py                   # format entire file
```

Instead of individual file names, you can specify directories to darker, and it will find the changed files. However, if a directory is large, like a clone of the LLVM repository, darker can be painfully slow. In that case, you might wish to use git to list changed files. For example:

```
$ darker -r HEAD^ $(git diff --name-only --diff-filter=d HEAD^)
```

Mechanical Source Issues#

Source Code Formatting#

Commenting#

Comments are important for readability and maintainability. When writing comments, write them as English prose, using proper capitalization, punctuation, etc. Aim to describe what the code is trying to do and why, not *how* it does it at a micro level. Here are a few important things to document:

File Headers#

Every source file should have a header on it that describes the basic purpose of the file. The standard header looks like this:

```
//====-----
//
// Part of the LLVM Project, under the Apache License v2.0 with LLVM Exceptions.
// See https://llvm.org/LICENSE.txt for license information.
// SPDX-License-Identifier: Apache-2.0 WITH LLVM-exception
//
//====-----
///
/// \file
/// This file contains the declaration of the Instruction class, which is the
/// base class for all of the VM instructions.
///
//====-----
```

The first section in the file is a concise note that defines the license that the file is released under. This makes it perfectly clear what terms the source code can be distributed under and should not be modified in any way.

The main body is a [Doxygen](#) comment (identified by the `///` comment marker instead of the usual `/**`) describing the purpose of the file. The first sentence (or a passage beginning with `\brief`) is used as an abstract. Any additional information should be separated by a blank line. If an algorithm is based on a paper or is described in another source, provide a reference.

Header Guard#

The header file's guard should be the all-caps path that a user of this header would `#include`, using `'_'` instead of path separator and extension marker. For example, the header file `llvm/include/llvm/Analysis/Utils/Local.h` would be `#include`-ed as `#include "llvm/Analysis/Utils/Local.h"`, so its guard is `LLVM_ANALYSIS_UTILS_LOCAL_H`.

Class overviews#

Classes are a fundamental part of an object-oriented design. As such, a class definition should have a comment block that explains what the class is used for and how it works. Every non-trivial class is expected to have a `doxygen` comment block.

Method information#

Methods and global functions should also be documented. A quick note about what it does and a description of the edge cases is all that is necessary here. The reader should be able to understand how to use interfaces without reading the code itself.

Good things to talk about here are what happens when something unexpected happens, for instance, does the method return null?

Comment Formatting#

In general, prefer C++-style comments (`//` for normal comments, `///` for `doxygen` documentation comments). There are a few cases when it is useful to use C-style (`/* */`) comments, however:

1. When writing C code to be compatible with C89.
2. When writing a header file that may be `#included` by a C source file.
3. When writing a source file that is used by a tool that only accepts C-style comments.
4. When documenting the significance of constants used as actual parameters in a call. This is most helpful for `bool` parameters, or passing `0` or `nullptr`. The comment should contain the parameter name, which ought to be meaningful. For example, it's not clear what the parameter means in this call:

```
Object.emitName(nullptr);
```

An in-line C-style comment makes the intent obvious:

```
Object.emitName(/*Prefix=*/nullptr);
```

Commenting out large blocks of code is discouraged, but if you really have to do this (for documentation purposes or as a suggestion for debug printing), use `#if 0` and `#endif`. These nest properly and are better behaved in general than C-style comments.

Doxygen Use in Documentation Comments#

Use the `\file` command to turn the standard file header into a file-level comment.

Include descriptive paragraphs for all public interfaces (public classes, member and non-member functions). Avoid restating the information that can be inferred from the API name or signature. The first sentence (or a paragraph beginning with `\brief`) is used as an abstract. Try to use a single sentence as the `\brief` adds visual clutter. Put detailed discussion into separate paragraphs.

A minimal documentation comment:

```
/// Sets the xyzzy property to \p Baz.  
void setXyzzy(bool Baz);
```

Only include code examples, function parameters and return values when it provides additional information, such as intent, usage, or behavior that's non-obvious. Use descriptive function and argument names to eliminate the need for documentation comments when possible.

To refer to parameter names inside a paragraph, use the `\p name` command. Don't use the `\arg name` command since it starts a new paragraph that contains documentation for the parameter.

Wrap non-inline code examples in `\code ... \endcode`.

To document a function parameter, start a new paragraph with the `\param name` command. If the parameter is used as an out or an in/out parameter, use the `\param [out] name` or `\param [in,out] name` command, respectively.

To describe function return value, start a new paragraph with the `\returns` command.

A documentation comment that uses all Doxygen features in a preferred way:

```
/// Does foo and bar.
///
/// Does not do foo the usual way if \p Baz is true.
///
/// Typical usage:
/// \code
///   fooBar(false, "quux", Res);
/// \endcode
///
/// \param Quux kind of foo to do.
/// \param [out] Result filled with bar sequence on foo success.
///
/// \returns true on success.
bool fooBar(bool Baz, StringRef Quux, std::vector<int> &Result);
```

Don't duplicate the documentation comment in the header file and in the implementation file. Put the documentation comments for public APIs into the header file. Documentation comments for private APIs can go to the implementation file. In any case, implementation files can include additional comments (not necessarily in Doxygen markup) to explain implementation details as needed.

Don't duplicate the function or class name at the beginning of the comment. For humans, it is obvious which function or class is being documented; automatic documentation processing tools are smart enough to bind the comment to the correct declaration.

Avoid:

```
// Example.h:
// example - Does something important.
void example();

// Example.cpp:
// example - Does something important.
void example() { ... }
```

Preferred:

```
// Example.h:
/// Does something important.
void example();

// Example.cpp:
/// Builds a B-tree in order to do foo. See paper by...
void example() { ... }
```

Error and Warning Messages#

Clear diagnostic messages are important to help users identify and fix issues in their inputs. Use succinct but correct English prose that gives the user the context needed to understand what went wrong. Also, to match error message styles commonly produced by other tools, start the first sentence with a lowercase letter, and finish the last sentence without a period, if it would end in one otherwise. Sentences which end with different punctuation, such as “did you forget ‘;’?”, should still do so.

For example, this is a good error message:

```
error: file.o: section header 3 is corrupt. Size is 10 when it should be 20
```

This is a bad message, since it does not provide useful information and uses the wrong style:

```
error: file.o: Corrupt section header.
```

As with other coding standards, individual projects, such as the Clang Static Analyzer, may have preexisting styles that do not conform to this. If a different formatting scheme is used consistently throughout the project, use that style instead. Otherwise, this standard applies to all LLVM tools, including clang, clang-tidy, and so on.

If the tool or project does not have existing functions to emit warnings or errors, use the error and warning handlers provided in `Support/WithColor.h` to ensure they are printed in the appropriate style, rather than printing to `stderr` directly.

When using `report_fatal_error`, follow the same standards for the message as regular error messages. Assertion messages and `llvm_unreachable` calls do not necessarily need to follow these same styles as they are automatically formatted, and thus these guidelines may not be suitable.

#include Style#

Immediately after the [header file comment](#) (and include guards if working on a header file), the [minimal list of #includes](#) required by the file should be listed. We prefer these `#includes` to be listed in this order:

1. Main Module Header
2. Local/Private Headers
3. LLVM project/subproject headers (`clang/...`, `lldb/...`, `llvm/...`, etc)
4. System `#includes`

and each category should be sorted lexicographically by the full path.

The [Main Module Header](#) file applies to `.cpp` files which implement an interface defined by a `.h` file. This `#include` should always be included **first** regardless of where it lives on the file system. By including a header file first in the `.cpp` files that implement the interfaces, we ensure that the header does not have any hidden dependencies which are not explicitly `#included` in the header, but should be. It is also a form of documentation in the `.cpp` file to indicate where the interfaces it implements are defined.

LLVM project and subproject headers should be grouped from most specific to least specific, for the same reasons described above. For example, LLDB depends on both clang and LLVM, and clang depends on LLVM. So an LLDB source file should include `lldb` headers first, followed by `clang` headers, followed by `llvm` headers, to reduce the possibility (for example) of an LLDB header accidentally picking up a missing include due to the previous inclusion of that header in the main source

file or some earlier header file. clang should similarly include its own headers before including llvm headers. This rule applies to all LLVM subprojects.

Source Code Width#

Write your code to fit within 80 columns.

There must be some limit to the width of the code in order to allow developers to have multiple files side-by-side in windows on a modest display. If you are going to pick a width limit, it is somewhat arbitrary, but you might as well pick something standard. Going with 90 columns (for example) instead of 80 columns wouldn't add any significant value and would be detrimental to printing out code. Also many other projects have standardized on 80 columns, so some people have already configured their editors for it (vs something else, like 90 columns).

However, documentation files are not source code files, and instead of fitting into 80 columns, they must be formatted to one sentence per line. This way a change in the middle of a paragraph doesn't cause unnecessary changes in subsequent lines, making it easier for reviewers to see what has changed when documentation is updated.

Another exception are TableGen files (.td). There are no tools that can parse, verify and edit them to match our coding style, including clang-format, so they are excluded from this limit. It is still good practice to try to fit them into 80 columns, but it is not a requirement. As with other changes, if you patch a TD file, do not reformat the entire file, but only the lines you are changing.

If your reformatting is causing unnecessary changes in subsequent lines, please just follow the existing formatting. Unnecessary changes in TD files cause unnecessary churn in the file history and force recompilation of many unnecessary files. It also causes unnecessary changes in forks working on the same TD files, which makes it harder to rebase and merge later.

Whitespace#

In all cases, prefer spaces to tabs in source files. People have different preferred indentation levels, and different styles of indentation that they like; this is fine. What isn't fine is that different editors/viewers expand tabs out to different tab stops. This can cause your code to look completely unreadable, and it is not worth dealing with.

As always, follow the [Golden Rule](#) above: follow the style of existing code if you are modifying and extending it.

Do not add trailing whitespace. Some common editors will automatically remove trailing whitespace when saving a file which causes unrelated changes to appear in diffs and commits.

Format Lambdas Like Blocks Of Code#

When formatting a multi-line lambda, format it like a block of code. If there is only one multi-line lambda in a statement, and there are no expressions lexically after it in the statement, drop the indent to the standard two space indent for a block of code, as if it were an if-block opened by the preceding part of the statement:

```
std::sort(foo.begin(), foo.end(), [&](Foo a, Foo b) -> bool {
    if (a.blah < b.blah)
        return true;
    if (a.baz < b.baz)
        return true;
    return a.bam < b.bam;
});
```

To take best advantage of this formatting, if you are designing an API which accepts a continuation or

single callable argument (be it a function object, or a `std::function`), it should be the last argument if at all possible.

If there are multiple multi-line lambdas in a statement, or additional parameters after the lambda, indent the block two spaces from the indent of the `[]`:

```
dyn_switch(V->stripPointerCasts(),
    [] (PHINode *PN) {
        // process phis...
    },
    [] (SelectInst *SI) {
        // process selects...
    },
    [] (LoadInst *LI) {
        // process loads...
    },
    [] (AllocaInst *AI) {
        // process allocas...
    });
```

Braced Initializer Lists#

Starting from C++11, there are significantly more uses of braced lists to perform initialization. For example, they can be used to construct aggregate temporaries in expressions. They now have a natural way of ending up nested within each other and within function calls in order to build up aggregates (such as option structs) from local variables.

The historically common formatting of braced initialization of aggregate variables does not mix cleanly with deep nesting, general expression contexts, function arguments, and lambdas. We suggest new code use a simple rule for formatting braced initialization lists: act as if the braces were parentheses in a function call. The formatting rules exactly match those already well understood for formatting nested function calls. Examples:

```
foo({a, b, c}, {1, 2, 3});

llvm::Constant *Mask[] = {
    llvm::ConstantInt::get(llvm::Type::getInt32Ty(getLLVMContext()), 0),
    llvm::ConstantInt::get(llvm::Type::getInt32Ty(getLLVMContext()), 1),
    llvm::ConstantInt::get(llvm::Type::getInt32Ty(getLLVMContext()), 2)};
```

This formatting scheme also makes it particularly easy to get predictable, consistent, and automatic formatting with tools like [Clang Format](#).

Language and Compiler Issues#

Treat Compiler Warnings Like Errors#

Compiler warnings are often useful and help improve the code. Those that are not useful, can be often suppressed with a small code change. For example, an assignment in the `if` condition is often a typo:

```
if (V = getValue()) {
    ...
}
```

Several compilers will print a warning for the code above. It can be suppressed by adding parentheses:

```
if ((V = getValue())) {
    ...
}
```

Write Portable Code#

In almost all cases, it is possible to write completely portable code. When you need to rely on non-portable code, put it behind a well-defined and well-documented interface.

Do not use RTTI or Exceptions#

In an effort to reduce code and executable size, LLVM does not use exceptions or RTTI ([runtime type information](#), for example, `dynamic_cast<>`).

That said, LLVM does make extensive use of a hand-rolled form of RTTI that use templates like [isa<>](#), [cast<>](#), and [dyn_cast<>](#). This form of RTTI is opt-in and can be [added to any class](#).

Prefer C++-style casts#

When casting, use `static_cast`, `reinterpret_cast`, and `const_cast`, rather than C-style casts. There are two exceptions to this:

- When casting to `void` to suppress warnings about unused variables (as an alternative to `[[maybe_unused]]`). Prefer C-style casts in this instance. Note that if the variable is unused because it's used only in `assert`, use `[[maybe_unused]]` instead of a C-style void cast.
- When casting between integral types (including enums that are not strongly-typed), functional-style casts are permitted as an alternative to `static_cast`.

Do not use Static Constructors#

Static constructors and destructors (e.g., global variables whose types have a constructor or destructor) should not be added to the code base, and should be removed wherever possible. Global constants (including static variables defined in functions) should be `constexpr` where possible to avoid inadvertant startup overhead.

```
// Avoid: this will cause the string to be constructed in a global constructor
// and be destructed in a global destructor. The same applies for global/static
// maps, sets, etc.
static std::string VeryBad = "abc123";

struct BadWrapInt {
    int x;
    BadWrapInt(int x) : x(x) {}
};
// Avoid: the BadWrapInt constructor is not constexpr and this will cause the
// initialization of Bad in a global constructor.
static const BadWrapInt Bad(1);

struct GoodWrapInt {
    int x;
    constexpr GoodWrapInt(int x) : x(x) {}
};
// Ok.
static constexpr GoodWrapInt Good(1);
```

Globals in different source files are initialized in an [arbitrary order](#), making the code more difficult to reason about.

Static constructors have a negative impact on the launch time of programs that use LLVM as a library. We would really like for there to be zero cost for linking in an additional LLVM target or other library into an application, but static constructors undermine this goal.

Avoid Pointers in Global/Static Constants#

Static constants that contain non-null pointers should be avoided, especially for large or frequently used data structures.

All static constants that contain pointers need to be relocated on every startup when LLVM is built as position-independent code, increasing startup times and memory usage.

```
// Avoid:StringRef stores pointers to string; this array needs to be touched
// on every startup (typically 32 bytes).
static constexpr StringRef Bad1[] = {"abc", "def"};

// Avoid: likewise (typically 16 bytes).
static const char *const Bad2[] = {"abc", "def"};

// Avoid: this is a mutable array!
static const char *ReallyBad[] = {"abc", "def"};

// Ok.
static constexpr char OkStorage[] = "\0abc\0def";
StringRef getOk(StringTable::Offset Off) { return StringTable(OkStorage)[Off]; }
// Avoid: StringTable itself is just a wrapper around StringRef.
extern constexpr StringTable Bad3(OkStorage);

// Avoid.
struct KeyValue {
    const char *Key;
    unsigned Value;
};
static constexpr KeyValue KVs[] = {"abc", 1}, {"def", 2}};

// Ok.
constexpr EnumStringDef<unsigned> KVDefs[] = {"abc", 1}, {"def", 2}};
static constexpr auto KVs = BUILD_ENUM_STRINGS(KVDefs);
```

Use of `class` and `struct` Keywords#

In C++, the `class` and `struct` keywords can be used almost interchangeably. The only difference is when they are used to declare a class: `class` makes all members private by default while `struct` makes all members public by default.

- All declarations and definitions of a given `class` or `struct` must use the same keyword. For example:

```
// Avoid if `Example` is defined as a struct.
class Example;

// OK.
struct Example;

struct Example { ... };
```

- `struct` should be used when *all* members are declared public.

```
// Avoid using `struct` here, use `class` instead.
struct Foo {
private:
    int Data;
public:
    Foo() : Data(0) { }
    int getData() const { return Data; }
    void setData(int D) { Data = D; }
};

// OK to use `struct`: all members are public.
```

```
struct Bar {
    int Data;
    Bar() : Data(0) { }
};
```

Do not use Braced Initializer Lists to Call a Constructor#

Starting from C++11 there is a “generalized initialization syntax” which allows calling constructors using braced initializer lists. Do not use these to call constructors with non-trivial logic or if you care that you’re calling some *particular* constructor. Those should look like function calls using parentheses rather than like aggregate initialization. Similarly, if you need to explicitly name the type and call its constructor to create a temporary, don’t use a braced initializer list. Instead, use a braced initializer list (without any type for temporaries) when doing aggregate initialization or something notionally equivalent. Examples:

```
class Foo {
public:
    // Construct a Foo by reading data from the disk in the whizbang format, ...
    Foo(std::string filename);

    // Construct a Foo by looking up the Nth element of some global data ...
    Foo(int N);

    // ...
};

// The Foo constructor call is reading a file, don't use braces to call it.
llvm::fill(foo, Foo("name"));

// The pair is being constructed like an aggregate, use braces.
bar_map.insert({my_key, my_value});
```

If you use a braced initializer list when initializing a variable, use an equals before the open curly brace:

```
int data[] = {0, 1, 2, 3};
```

Use auto Type Deduction to Make Code More Readable#

Some are advocating a policy of “almost always `auto`” in C++11; however, LLVM uses a more moderate stance. Use `auto` if and only if it makes the code more readable or easier to maintain. Don’t “almost always” use `auto`, but do use `auto` with initializers like `cast<Foo>(...)` or other places where the type is already obvious from the context. Another time when `auto` works well for these purposes is when the type would have been abstracted away anyway, often behind a container’s typedef such as `std::vector<T>::iterator`.

Similarly, C++14 adds generic lambda expressions where parameter types can be `auto`. Use these where you would have used a template.

Beware unnecessary copies with auto#

The convenience of `auto` makes it easy to forget that its default behavior is a copy. Particularly in range-based `for` loops, careless copies are expensive.

Use `auto &` for values and `auto *` for pointers unless you need to make a copy.

```
// Typically there's no reason to copy.
for (const auto &Val : Container) observe(Val);
for (auto &Val : Container) Val.change();

// Remove the reference if you really want a new copy.
for (auto Val : Container) { Val.change(); saveSomewhere(Val); }
```

```
// Copy pointers, but make it clear that they're pointers.
for (const auto *Ptr : Container) observe(*Ptr);
for (auto *Ptr : Container) Ptr->change();
```

Beware of non-determinism due to ordering of pointers#

In general, there is no relative ordering among pointers. As a result, when unordered containers like sets and maps are used with pointer keys the iteration order is undefined. Hence, iterating such containers may result in non-deterministic code generation. While the generated code might work correctly, non-determinism can make it harder to reproduce bugs and debug the compiler.

In case an ordered result is expected, remember to sort an unordered container before iteration. Or use ordered containers like `vector/MapVector/SetVector` if you want to iterate pointer keys.

Beware of non-deterministic sorting order of equal elements#

`std::sort` uses a non-stable sorting algorithm in which the order of equal elements is not guaranteed to be preserved. Thus using `std::sort` for a container having equal elements may result in non-deterministic behavior. To uncover such instances of non-determinism, LLVM has introduced a new `llvm::sort` wrapper function. For an `EXPENSIVE_CHECKS` build this will randomly shuffle the container before sorting. Default to using `llvm::sort` instead of `std::sort`.

Style Issues#

The High-Level Issues#

Self-contained Headers#

Header files should be self-contained (compile on their own) and end in `.h`. Non-header files that are meant for inclusion should end in `.inc` and be used sparingly.

All header files should be self-contained. Users and refactoring tools should not have to adhere to special conditions to include the header. Specifically, a header should have header guards and include all other headers it needs.

There are rare cases where a file designed to be included is not self-contained. These are typically intended to be included at unusual locations, such as the middle of another file. They might not use header guards, and might not include their prerequisites. Name such files with the `.inc` extension. Use sparingly, and prefer self-contained headers when possible.

In general, a header should be implemented by one or more `.cpp` files. Each of these `.cpp` files should include the header that defines their interface first. This ensures that all of the dependencies of the header have been properly added to the header itself, and are not implicit. System headers should be included after user headers for a translation unit.

Library Layering#

A directory of header files (for example, `include/llvm/Foo`) defines a library (`Foo`). One library (both its headers and implementation) should only use things from the libraries listed in its dependencies.

Some of this constraint can be enforced by classic Unix linkers (Mac & Windows linkers, as well as `lld`, do not enforce this constraint). A Unix linker searches left to right through the libraries specified on its command line and never revisits a library. In this way, no circular dependencies between libraries can exist.

This doesn't fully enforce all inter-library dependencies, and importantly doesn't enforce header file

circular dependencies created by inline functions. A good way to answer the “is this layered correctly” would be to consider whether a Unix linker would succeed at linking the program if all inline functions were defined out-of-line. (& for all valid orderings of dependencies — since linking resolution is linear, it’s possible that some implicit dependencies can sneak through: A depends on B and C, so valid orderings are “C B A” or “B C A”, in both cases the explicit dependencies come before their use. But in the first case, B could still link successfully if it implicitly depended on C, or the opposite in the second case)

#include as Little as Possible#

`#include` hurts compile time performance. Don’t do it unless you have to, especially in header files.

But wait! Sometimes you need to have the definition of a class to use it, or to inherit from it. In these cases go ahead and `#include` that header file. Be aware, however, that there are many cases where you don’t need to have the full definition of a class. If you are using a pointer or reference to a class, you don’t need the header file. If you are simply returning a class instance from a prototyped function or method, you don’t need it. In fact, for most cases, you simply don’t need the definition of a class. And not `#include`ing speeds up compilation.

It is easy to try to go overboard on this recommendation, however. You **must** include all of the header files that you are using — you can include them either directly or indirectly through another header file. To make sure that you don’t accidentally forget to include a header file in your module header, make sure to include your module header **first** in the implementation file (as mentioned above). This way there won’t be any hidden dependencies that you’ll find out about later.

Keep “Internal” Headers Private#

Many modules have a complex implementation that causes them to use more than one implementation (.cpp) file. It is often tempting to put the internal communication interface (helper classes, extra functions, etc) in the public module header file. Don’t do this!

If you really need to do something like this, put a private header file in the same directory as the source files, and include it locally. This ensures that your private interface remains private and undisturbed by outsiders.

Note

It’s okay to put extra implementation methods in a public class itself. Just make them private (or protected) and all is well.

Use Namespace Qualifiers to Define Previously Declared Symbols#

When providing an out-of-line definition for various symbols (variables, functions, opaque classes) in a source file, do not open namespace blocks in the source file. Instead, use namespace qualifiers to help ensure that your definition matches an existing declaration. Do this:

```
// Foo.h
namespace llvm {
extern int FooVal;
int foo(const char *s);

namespace detail {
class FooImpl;
} // namespace detail
} // namespace llvm

// Foo.cpp
#include "Foo.h"
using namespace llvm;
```

```
int llvm::FooVal;

int llvm::foo(const char *s) {
    // ...
}

class detail::FooImpl {
    // ...
}
```

Doing this helps to avoid bugs where the definition does not match the declaration from the header. For example, the following C++ code defines a new overload of `llvm::foo` instead of providing a definition for the existing function declared in the header:

```
// Foo.cpp
#include "Foo.h"
namespace llvm {
int foo(char *s) { // Mismatch between "const char *" and "char *"
}
} // namespace llvm
```

This error will not be caught until the build is nearly complete, when the linker fails to find a definition for any uses of the original function. If the function were instead defined with a namespace qualifier, the error would have been caught immediately when the definition was compiled.

Class method implementations must already name the class and new overloads cannot be introduced out of line, so this recommendation does not apply to them.

Use Early Exits and `continue` to Simplify Code

When reading code, keep in mind how much state and how many previous decisions have to be remembered by the reader to understand a block of code. Aim to reduce indentation where possible when it doesn't make it more difficult to understand the code. One great way to do this is by making use of early exits and the `continue` keyword in long loops. Consider this code that does not use an early exit:

```
Value *doSomething(Instruction *I) {
    if (!I->isTerminator() &&
        I->hasOneUse() && doOtherThing(I)) {
        ... some long code ....
    }

    return 0;
}
```

This code has several problems if the body of the `if` is large. When you're looking at the top of the function, it isn't immediately clear that this *only* does interesting things with non-terminator instructions, and only applies to things with the other predicates. Second, it is relatively difficult to describe (in comments) why these predicates are important because the `if` statement makes it difficult to lay out the comments. Third, when you're deep within the body of the code, it is indented an extra level. Finally, when reading the top of the function, it isn't clear what the result is if the predicate isn't true; you have to read to the end of the function to know that it returns null.

It is much preferred to format the code like this:

```
Value *doSomething(Instruction *I) {
    // Terminators never need 'something' done to them because ...
    if (I->isTerminator())
        return 0;

    // We conservatively avoid transforming instructions with multiple uses
    // because goats like cheese.
```

```

if (!I->hasOneUse())
    return 0;

// This is really just here for example.
if (!doOtherThing(I))
    return 0;

... some long code ....
}

```

This fixes these problems. A similar problem frequently happens in `for` loops. A silly example is something like this:

```

for (Instruction &I : BB) {
    if (auto *BO = dyn_cast<BinaryOperator>(&I)) {
        Value *LHS = BO->getOperand(0);
        Value *RHS = BO->getOperand(1);
        if (LHS != RHS) {
            ...
        }
    }
}

```

When you have very, very small loops, this sort of structure is fine. But if it exceeds more than 10-15 lines, it becomes difficult for people to read and understand at a glance. The problem with this sort of code is that it gets very nested very quickly. This means that the reader of the code has to keep a lot of context in their brain to remember what is going immediately on in the loop, because they don't know if/when the `if` conditions will have `elses` etc. It is strongly preferred to structure the loop like this:

```

for (Instruction &I : BB) {
    auto *BO = dyn_cast<BinaryOperator>(&I);
    if (!BO) continue;

    Value *LHS = BO->getOperand(0);
    Value *RHS = BO->getOperand(1);
    if (LHS == RHS) continue;

    ...
}

```

This has all the benefits of using early exits for functions: it reduces the nesting of the loop, it makes it easier to describe why the conditions are true, and it makes it obvious to the reader that there is no `else` coming up that they have to push context into their brain for. If a loop is large, this can be a big understandability win.

Don't use `else` after a `return`<#>

For similar reasons as above (reduction of indentation and easier reading), please do not use `'else'` or `'else if'` after something that interrupts control flow — like `return`, `break`, `continue`, `goto`, etc. For example:

```

case 'J': {
    if (Signed) {
        Type = Context.getsigjmp_bufType();
        if (Type.isNull()) {
            Error = ASTContext::GE_Missing_sigjmp_buf;
            return QualType();
        } else {
            break; // Unnecessary.
        }
    } else {
        Type = Context.getjmp_bufType();
        if (Type.isNull()) {

```

```

        Error = ASTContext::GE_Missing_jump_buf;
        return QualType();
    } else {
        break; // Unnecessary.
    }
}
}

```

It is better to write it like this:

```

case 'J':
    if (Signed) {
        Type = Context.getsigjmp_bufType();
        if (Type.isNull()) {
            Error = ASTContext::GE_Missing_sigjmp_buf;
            return QualType();
        }
    } else {
        Type = Context.getjmp_bufType();
        if (Type.isNull()) {
            Error = ASTContext::GE_Missing_jump_buf;
            return QualType();
        }
    }
    break;

```

Or better yet (in this case) as:

```

case 'J':
    if (Signed)
        Type = Context.getsigjmp_bufType();
    else
        Type = Context.getjmp_bufType();

    if (Type.isNull()) {
        Error = Signed ? ASTContext::GE_Missing_sigjmp_buf :
                        ASTContext::GE_Missing_jump_buf;
        return QualType();
    }
    break;

```

The idea is to reduce indentation and the amount of code you have to keep track of when reading the code.

Note: this advice does not apply to a `constexpr if` statement. The substatement of the `else` clause may be a discarded statement, so removing the `else` can cause unexpected template instantiations. Thus, the following example is correct:

```

template<typename T>
static constexpr bool VarTempl = true;

template<typename T>
int func() {
    if constexpr (VarTempl<T>)
        return 1;
    else
        static_assert(!VarTempl<T>);
}

```

Turn Predicate Loops into Predicate Functions#

It is very common to write small loops that just compute a boolean value. There are a number of ways that people commonly write these, but an example of this sort of thing is:

```

bool FoundFoo = false;
for (unsigned I = 0, E = BarList.size(); I != E; ++I)
    if (BarList[I]->isFoo()) {
        FoundFoo = true;
        break;
    }

if (FoundFoo) {
    ...
}

```

Instead of this sort of loop, we prefer to use a predicate function (which may be [static](#)) that uses [early exits](#):

```

/// \returns true if the specified list has an element that is a foo.
static bool containsFoo(const std::vector<Bar*> &List) {
    for (unsigned I = 0, E = List.size(); I != E; ++I)
        if (List[I]->isFoo())
            return true;
    return false;
}
...

if (containsFoo(BarList)) {
    ...
}

```

There are many reasons for doing this: it reduces indentation and factors out code which can often be shared by other code that checks for the same predicate. More importantly, it *forces you to pick a name* for the function, and forces you to write a comment for it. In this silly example, this doesn't add much value. However, if the condition is complex, this can make it a lot easier for the reader to understand the code that queries for this predicate. Instead of being faced with the in-line details of how we check to see if the BarList contains a foo, we can trust the function name and continue reading with better locality.

The Low-Level Issues#

Name Types, Functions, Variables, and Enumerators Properly#

Poorly-chosen names can mislead the reader and cause bugs. We cannot stress enough how important it is to use *descriptive* names. Pick names that match the semantics and role of the underlying entities, within reason. Avoid abbreviations unless they are well known. After picking a good name, make sure to use consistent capitalization for the name, as inconsistency requires clients to either memorize the APIs or to look it up to find the exact spelling.

In general, names should be in camel case (e.g. `TextFileReader` and `isLValue()`). Different kinds of declarations have different rules:

- **Type names** (including classes, structs, enums, typedefs, etc) should be nouns and start with an upper-case letter (e.g. `TextFileReader`).
- **Variable names** should be nouns (as they represent state). The name should be camel case, and start with an upper-case letter (e.g. `Leader` or `Boats`).
- **Function names** should be verb phrases (as they represent actions), and command-like function should be imperative. The name should be camel case, and start with a lowercase letter (e.g. `openFile()` or `isFoo()`).
- **Enum declarations** (e.g. `enum Foo { ... }`) are types, so they should follow the naming conventions for types. A common use for enums is as a discriminator for a union, or an indicator of a subclass. When an enum is used for something like this, it should have a `Kind` suffix (e.g.

ValueKind).

- **Enumerators** (e.g. `enum { Foo, Bar }`) and **public member variables** should start with an upper-case letter, just like types. Unless the enumerators are defined in their own small namespace or inside a class, enumerators should have a prefix corresponding to the enum declaration name. For example, `enum ValueKind { ... }`; may contain enumerators like `VK_Argument`, `VK_BasicBlock`, etc. Enumerators that are just convenience constants are exempt from the requirement for a prefix. For instance:

```
enum {
    MaxSize = 42,
    Density = 12
};
```

As an exception, classes that mimic STL classes can have member names in STL's style of lowercase words separated by underscores (e.g. `begin()`, `push_back()`, and `empty()`). Classes that provide multiple iterators should add a singular prefix to `begin()` and `end()` (e.g. `global_begin()` and `use_begin()`).

Here are some examples:

```
class VehicleMaker {
    ...
    Factory<Tire> F;           // Avoid: a non-descriptive abbreviation.
    Factory<Tire> Factory;     // Better: more descriptive.
    Factory<Tire> TireFactory; // Even better: if VehicleMaker has more than one
                             // kind of factories.
};

Vehicle makeVehicle(VehicleType Type) {
    VehicleMaker M;           // Might be OK if scope is small.
    Tire Tmp1 = M.makeTire(); // Avoid: 'Tmp1' provides no information.
    Light Headlight = M.makeLight("head"); // Good: descriptive.
    ...
}
```

Assert Liberally#

Use the “`assert`” macro to its fullest. Check all of your preconditions and assumptions. You never know when a bug (not necessarily even yours) might be caught early by an assertion, which reduces debugging time dramatically. The “`<cassert>`” header file is probably already included by the header files you are using, so it doesn't cost anything to use it.

To further assist with debugging, make sure to put some kind of error message in the assertion statement, which is printed if the assertion is tripped. This helps the poor debugger make sense of why an assertion is being made and enforced, and hopefully what to do about it. Here is one complete example:

```
inline Value *getOperand(unsigned I) {
    assert(I < Operands.size() && "getOperand() out of range!");
    return Operands[I];
}
```

Here are more examples:

```
assert(Ty->isPointerType() && "Can't allocate a non-pointer type!");
assert((Opcode == Shl || Opcode == Shr) && "ShiftInst Opcode invalid!");
assert(idx < getNumSuccessors() && "Successor # out of range!");
assert(V1.getType() == V2.getType() && "Constant types must be identical!");
```

```
assert(isa<PHINode>(Succ->front()) && "Only works on PHId BBs!");
```

You get the idea.

In the past, asserts were used to indicate a piece of code that should not be reached. These were typically of the form:

```
assert(0 && "Invalid radix for integer literal");
```

This has a few issues, the main one being that some compilers might not understand the assertion, or warn about a missing return in builds where assertions are compiled out.

Today, we have something much better: `llvm_unreachable`:

```
llvm_unreachable("Invalid radix for integer literal");
```

When assertions are enabled, this will print the message if it's ever reached and then exit the program. When assertions are disabled (i.e. in release builds), `llvm_unreachable` becomes a hint to compilers to skip generating code for this branch. If the compiler does not support this, it will fall back to the “abort” implementation.

Use `llvm_unreachable` to mark a specific point in code that should never be reached. This is especially desirable for addressing warnings about unreachable branches, etc., but can be used whenever reaching a particular code path is unconditionally a bug (not originating from user input; see below) of some kind. Use of `assert` should always include a testable predicate (as opposed to `assert(false)`).

If the error condition can be triggered by user input then the recoverable error mechanism described in [LLVM Programmer's Manual](#) should be used instead. In cases where this is not practical, `report_fatal_error` may be used.

Another issue is that values used only by assertions will produce an “unused value” warning when assertions are disabled. For example, this code will warn:

```
unsigned Size = V.size();
assert(Size > 42 && "Vector smaller than it should be");

bool NewToSet = Myset.insert(Value);
assert(NewToSet && "The value shouldn't be in the set yet");
```

These are two interesting different cases. In the first case, the call to `V.size()` is only useful for the `assert`, and we don't want it executed when assertions are disabled. Code like this should move the call into the `assert` itself. In the second case, the side effects of the call must happen whether the `assert` is enabled or not. In this case, the value should be defined using the `[[maybe_unused]]` attribute to suppress the warning. To be specific, it is preferred to write the code like this:

```
assert(V.size() > 42 && "Vector smaller than it should be");

[[maybe_unused]] bool NewToSet = Myset.insert(Value);
assert(NewToSet && "The value shouldn't be in the set yet");
```

In C code where `[[maybe_unused]]` is not supported, use `void` cast to suppress an unused variable warning as follows:

```
LLVMValueRef Value = LLVMMetadataAsValue(Context, NodeMD);
assert(LLVMIsAValueAsMetadata(Value) != NULL);
(void)Value;
```

Do Not Use `using namespace std`<#>

In LLVM, we prefer to explicitly prefix all identifiers from the standard namespace with an “`std::`” prefix, rather than rely on “`using namespace std`”.

In header files, adding a `'using namespace XXX'` directive pollutes the namespace of any source file that `#includes` the header, creating maintenance issues.

In implementation files (e.g. `.cpp` files), the rule is more of a stylistic rule, but is still important. Basically, using explicit namespace prefixes makes the code **clearer**, because it is immediately obvious what facilities are being used and where they are coming from. And **more portable**, because namespace clashes cannot occur between LLVM code and other namespaces. The portability rule is important because different standard library implementations expose different symbols (potentially ones they shouldn't), and future revisions to the C++ standard will add more symbols to the `std` namespace. As such, we never use `'using namespace std;'` in LLVM.

The exception to the general rule (i.e. it's not an exception for the `std` namespace) is for implementation files. For example, all of the code in the LLVM project implements code that lives in the `'llvm'` namespace. As such, it is ok, and actually clearer, for the `.cpp` files to have a `'using namespace llvm;'` directive at the top, after the `#includes`. This reduces indentation in the body of the file for source editors that indent based on braces, and keeps the conceptual context cleaner. The general form of this rule is that any `.cpp` file that implements code in any namespace may use that namespace (and its parents'), but should not use any others.

Provide a Virtual Method Anchor for Classes in Headers<#>

If a class is defined in a header file and has a vtable (either it has virtual methods or it derives from classes with virtual methods), it must always have at least one out-of-line virtual method in the class. Without this, the compiler will copy the vtable and RTTI into every `.o` file that `#includes` the header, bloating `.o` file sizes and increasing link times.

Don't use default labels in fully covered switches over enumerations<#>

`-Wswitch` warns if a switch, without a default label, over an enumeration does not cover every enumeration value. If you write a default label on a fully covered switch over an enumeration then the `-Wswitch` warning won't fire when new elements are added to that enumeration. To help avoid adding these kinds of defaults, Clang has the warning `-Wcovered-switch-default` which is off by default but turned on when building LLVM with a version of Clang that supports the warning.

A knock-on effect of this stylistic requirement is that when building LLVM with GCC you may get warnings related to “control may reach end of non-void function” if you return from each case of a covered switch-over-enum because GCC assumes that the enum expression may take any representable value, not just those of individual enumerators. To suppress this warning, use `llvm_unreachable` after the switch.

Use range-based `for` loops wherever possible<#>

The introduction of range-based `for` loops in C++11 means that explicit manipulation of iterators is rarely necessary. We use range-based `for` loops wherever possible for all newly added code. For example:

```
BasicBlock *BB = ...
for (Instruction &I : *BB)
    ... use I ...
```

Usage of `std::for_each()/llvm::for_each()` functions is discouraged, unless the callable object already exists.

Don't evaluate `end()` every time through a loop#

In cases where range-based `for` loops can't be used and it is necessary to write an explicit iterator-based loop, pay close attention to whether `end()` is re-evaluated on each loop iteration. One common mistake is to write a loop in this style:

```
BasicBlock *BB = ...
for (auto I = BB->begin(); I != BB->end(); ++I)
    ... use I ...
```

The problem with this construct is that it evaluates “`BB->end()`” every time through the loop. Instead of writing the loop like this, we strongly prefer loops to be written so that they evaluate it once before the loop starts. A convenient way to do this is like so:

```
BasicBlock *BB = ...
for (auto I = BB->begin(), E = BB->end(); I != E; ++I)
    ... use I ...
```

The observant may quickly point out that these two loops may have different semantics: if the container (a basic block in this case) is being mutated, then “`BB->end()`” may change its value every time through the loop and the second loop may not in fact be correct. If you actually do depend on this behavior, please write the loop in the first form and add a comment indicating that you did it intentionally.

Why do we prefer the second form (when correct)? Writing the loop in the first form has two problems. First, it may be less efficient than evaluating it at the start of the loop. In this case, the cost is probably minor — a few extra loads every time through the loop. However, if the base expression is more complex, then the cost can rise quickly. I've seen loops where the end expression was actually something like: “`SomeMap[X]->end()`” and map lookups really aren't cheap. By writing it in the second form consistently, you eliminate the issue entirely and don't even have to think about it.

The second (even bigger) issue is that writing the loop in the first form hints to the reader that the loop is mutating the container (a fact that a comment would handily confirm!). If you write the loop in the second form, it is immediately obvious without even looking at the body of the loop that the container isn't being modified, which makes it easier to read the code and understand what it does.

While the second form of the loop is a few extra keystrokes, we do strongly prefer it.

`#include <iostream>` is Forbidden#

The use of `#include <iostream>` in library files is hereby **forbidden**, because many common implementations transparently inject a [static constructor](#) into every translation unit that includes it.

Note that using the other stream headers (`<sstream>` for example) is not problematic in this regard — just `<iostream>`. However, `raw_ostream` provides various APIs that are better performing for almost every use than `std::ostream` style APIs.

Note

New code should always use [raw_ostream](#) for writing, or the `llvm::MemoryBuffer` API for reading files.

Use `raw_ostream`#

LLVM includes a lightweight, simple, and efficient stream implementation in `llvm/Support/raw_ostream.h`, which provides all of the common features of `std::ostream`. All new code should use `raw_ostream` instead of `ostream`.

Unlike `std::ostream`, `raw_ostream` is not a template and can be forward declared as `class raw_ostream`. Public headers should generally not include the `raw_ostream` header, but use forward declarations and constant references to `raw_ostream` instances.

Avoid `std::endl`<#>

The `std::endl` modifier, when used with `ostreams` outputs a newline to the output stream specified. In addition to doing this, however, it also flushes the output stream. In other words, these are equivalent:

```
std::cout << std::endl;
std::cout << '\n' << std::flush;
```

Most of the time, you probably have no reason to flush the output stream, so it's better to use a literal `'\n'`.

Don't use `inline` when defining a function in a class definition<#>

A member function defined in a class definition is implicitly inline, so don't put the `inline` keyword in this case.

Don't:

```
class Foo {
public:
    inline void bar() {
        // ...
    }
};
```

Do:

```
class Foo {
public:
    void bar() {
        // ...
    }
};
```

Microscopic Details<#>

This section describes preferred low-level formatting guidelines along with reasoning on why we prefer them.

Spaces Before Parentheses<#>

Put a space before an open parenthesis only in control flow statements, but not in normal function call expressions and function-like macros. For example:

```
if (X) ...
for (I = 0; I != 100; ++I) ...
while (LLVMRocks) ...

somefunc(42);
assert(3 != 4 && "laws of math are failing me");

A = foo(42, 92) + bar(X);
```

The reason for doing this is not completely arbitrary. This style makes control flow operators stand out more, and makes expressions flow better.

Prefer Preincrement#

Hard fast rule: Preincrement ($++X$) may be no slower than postincrement ($X++$) and could very well be a lot faster than it. Use preincrementation whenever possible.

The semantics of postincrement include making a copy of the value being incremented, returning it, and then preincrementing the “work value”. For primitive types, this isn’t a big deal. But for iterators, it can be a huge issue (for example, some iterators contain stack and set objects in them... copying an iterator could invoke the copy ctor’s of these as well). In general, get in the habit of always using preincrement, and you won’t have a problem.

Namespace Indentation#

In general, we strive to reduce indentation wherever possible. This is useful because we want code to [fit into 80 columns](#) without excessive wrapping, but also because it makes it easier to understand the code. To facilitate this and avoid some insanely deep nesting on occasion, don’t indent namespaces. If it helps readability, feel free to add a comment indicating what namespace is being closed by a `}`. For example:

```
namespace llvm {
namespace knowledge {

/// This class represents things that Smith can have an intimate
/// understanding of and contains the data associated with it.
class Grokable {
...
public:
    explicit Grokable() { ... }
    virtual ~Grokable() = 0;

    ...
};

} // namespace knowledge
} // namespace llvm
```

Feel free to skip the closing comment when the namespace being closed is obvious for any reason. For example, the outer-most namespace in a header file is rarely a source of confusion. But namespaces both anonymous and named in source files that are being closed half way through the file probably could use clarification.

Restrict Visibility#

Functions and variables should have the most restricted visibility possible.

For class members, that means using appropriate `private`, `protected`, or `public` keyword to restrict their access.

For non-member functions, variables, and classes, that means restricting visibility to a single `.cpp` file if it is not referenced outside that file.

Visibility of file-scope non-member variables and functions can be restricted to the current translation unit by using either the `static` keyword or an anonymous namespace.

Anonymous namespaces are a great language feature that tells the C++ compiler that the contents of the namespace are only visible within the current translation unit, allowing more aggressive optimization and eliminating the possibility of symbol name collisions.

Anonymous namespaces are to C++ as `static` is to C functions and global variables. While `static` is available in C++, anonymous namespaces are more general: they can make entire classes private to a file.

The problem with anonymous namespaces is that they naturally want to encourage indentation of their body, and they reduce locality of reference: if you see a random function definition in a C++ file, it is easy to see if it is marked static, but seeing if it is in an anonymous namespace requires scanning a big chunk of the file.

Because of this, we have a simple guideline: make anonymous namespaces as small as possible, and only use them for class declarations. For example:

```
namespace {
class StringSort {
...
public:
    StringSort(...)
    bool operator<(const char *RHS) const;
};
} // namespace

static void runHelper() {
    ...
}

bool StringSort::operator<(const char *RHS) const {
    ...
}
```

Avoid putting declarations other than classes into anonymous namespaces:

```
namespace {
// ... many declarations ...

void runHelper() {
    ...
}

// ... many declarations ...
} // namespace
```

When you are looking at `runHelper` in the middle of a large C++ file, you have no immediate way to tell if this function is local to the file.

In contrast, when the function is marked static, you don't need to cross-reference faraway places in the file to tell that the function is local:

```
static void runHelper() {
    ...
}
```

Don't Use Braces on Simple Single-Statement Bodies of if/else/loop Statements#

When writing the body of an `if`, `else`, or `for/while` loop statement, we aim to reduce unnecessary line noise.

Omit braces when:

- The body consists of a single **simple** statement.
- The single statement is not preceded by a comment. (Hoist comments above the control statement if you can.)

- An `else` clause, if present, also meets the above criteria (single simple statement, no associated comments).

Use braces in all other cases, including:

- Multi-statement bodies
- Single-statement bodies with non-hoistable comments
- Complex single-statement bodies (e.g., deep nesting, complex nested loops)
- Inconsistent bracing within `if/else if/else` chains (if one block requires braces, all must)
- `if` statements ending with a nested `if` lacking an `else` (to prevent “dangling else”)

The examples below provide guidelines for these cases:

```
// Omit the braces since the body is simple and clearly associated with the
// `if`.
if (isa<FunctionDecl>(D))
    handleFunctionDecl(D);
else if (isa<VarDecl>(D))
    handleVarDecl(D);

// Here we document the condition itself and not the body.
if (isa<VarDecl>(D)) {
    // It is necessary that we explain the situation with this surprisingly long
    // comment, so it would be unclear without the braces whether the following
    // statement is in the scope of the `if`.
    // Because the condition is documented, we can't really hoist this
    // comment that applies to the body above the `if`.
    handleOtherDecl(D);
}

// Use braces on the outer `if` to avoid a potential dangling `else`
// situation.
if (isa<VarDecl>(D)) {
    if (shouldProcessAttr(A))
        handleAttr(A);
}

// Use braces for the `if` block to keep it uniform with the `else` block.
if (isa<FunctionDecl>(D)) {
    handleFunctionDecl(D);
} else {
    // In this `else` case, it is necessary that we explain the situation with
    // this surprisingly long comment, so it would be unclear without the braces
    // whether the following statement is in the scope of the `if`.
    handleOtherDecl(D);
}

// Use braces for the `else if` and `else` block to keep it uniform with the
// `if` block.
if (isa<FunctionDecl>(D)) {
    verifyFunctionDecl(D);
    handleFunctionDecl(D);
} else if (isa<GlobalVarDecl>(D)) {
    handleGlobalVarDecl(D);
} else {
    handleOtherDecl(D);
}

// This should also omit braces. The `for` loop contains only a single
// statement, so it shouldn't have braces. The `if` also only contains a
// single simple statement (the `for` loop), so it also should omit braces.
```



```

if (isa<FunctionDecl>(D))
    for (auto *A : D.attrs())
        handleAttr(A);

// Use braces for a `do-while` loop and its enclosing statement.
if (Tok->is(tok::l_brace)) {
    do {
        Tok = Tok->Next;
    } while (Tok);
}

// Use braces for the outer `if` since the nested `for` is braced.
if (isa<FunctionDecl>(D)) {
    for (auto *A : D.attrs()) {
        // In this `for` loop body, it is necessary that we explain the situation
        // with this surprisingly long comment, forcing braces on the `for` block.
        handleAttr(A);
    }
}

// Use braces on the outer block because there are more than two levels of
// nesting.
if (isa<FunctionDecl>(D)) {
    for (auto *A : D.attrs())
        for (ssize_t i : llvm::seq<ssize_t>(count))
            handleAttrOnDecl(D, A, i);
}

// Use braces on the outer block because of a nested `if`; otherwise, the
// compiler would warn: `add explicit braces to avoid dangling else`
if (auto *D = dyn_cast<FunctionDecl>(D)) {
    if (shouldProcess(D))
        handleVarDecl(D);
    else
        markAsIgnored(D);
}

```

Use Unix line endings for files#

Use Unix line endings for all files. CRLF line endings are allowed as an exception for test files that intend to test CRLF handling or when the file format requires it (like .bat or .rc files).

See Also#

A lot of these comments and recommendations have been culled from other sources. Two particularly important books for our work are:

1. [Effective C++](#) by Scott Meyers. Also interesting and useful are “More Effective C++” and “Effective STL” by the same author.
2. [Large-Scale C++ Software Design](#) by John Lakos

If you get some free time, and you haven’t read them: do so, you might learn something.

[Next](#)

[Bisecting LLVM code](#)

[Previous](#)

[LLVM Bug Life Cycle](#)

Copyright © 2003-2026, LLVM Project

Made with [Sphinx](#) and [@pradyunsg's Furo](#)

Last updated on 2026-08-02

On this page

- [LLVM Coding Standards](#)
 - [Introduction](#)
 - [Languages, Libraries, and Standards](#)
 - [C++ Standard Versions](#)
 - [C++ Standard Library](#)
 - [Python version and Source Code Formatting](#)
 - [Mechanical Source Issues](#)
 - [Source Code Formatting](#)
 - [Commenting](#)
 - [File Headers](#)
 - [Header Guard](#)
 - [Class overviews](#)
 - [Method information](#)
 - [Comment Formatting](#)
 - [Doxygen Use in Documentation Comments](#)
 - [Error and Warning Messages](#)
 - [#include Style](#)
 - [Source Code Width](#)
 - [Whitespace](#)
 - [Format Lambdas Like Blocks Of Code](#)
 - [Braced Initializer Lists](#)
 - [Language and Compiler Issues](#)
 - [Treat Compiler Warnings Like Errors](#)
 - [Write Portable Code](#)
 - [Do not use RTTI or Exceptions](#)
 - [Prefer C++-style casts](#)
 - [Do not use Static Constructors](#)
 - [Avoid Pointers in Global/Static Constants](#)
 - [Use of class and struct Keywords](#)
 - [Do not use Braced Initializer Lists to Call a Constructor](#)
 - [Use auto Type Deduction to Make Code More Readable](#)
 - [Beware unnecessary copies with auto](#)
 - [Beware of non-determinism due to ordering of pointers](#)
 - [Beware of non-deterministic sorting order of equal elements](#)
 - [Style Issues](#)
 - [The High-Level Issues](#)
 - [Self-contained Headers](#)
 - [Library Layering](#)
 - [#include as Little as Possible](#)
 - [Keep “Internal” Headers Private](#)
 - [Use Namespace Qualifiers to Define Previously Declared Symbols](#)
 - [Use Early Exits and continue to Simplify Code](#)
 - [Don't use else after a return](#)
 - [Turn Predicate Loops into Predicate Functions](#)
 - [The Low-Level Issues](#)
 - [Name Types, Functions, Variables, and Enumerators Properly](#)
 - [Assert Liberally](#)
 - [Do Not Use using namespace std](#)
 - [Provide a Virtual Method Anchor for Classes in Headers](#)
 - [Don't use default labels in fully covered switches over enumerations](#)
 - [Use range-based for loops wherever possible](#)
 - [Don't evaluate end\(\) every time through a loop](#)
 - [#include <iostream> is Forbidden](#)
 - [Use raw ostream](#)
 - [Avoid std::endl](#)
 - [Don't use inline when defining a function in a class definition](#)

- [Microscopic Details](#)
 - [Spaces Before Parentheses](#)
 - [Prefer Preincrement](#)
 - [Namespace Indentation](#)
 - [Restrict Visibility](#)
 - [Don't Use Braces on Simple Single-Statement Bodies of if/else/loop Statements](#)
 - [Use Unix line endings for files](#)
- [See Also](#)